

How Domain Models Are Useful in Object-Oriented Programming (OOP)

Introduction

- ▶ A domain model is a conceptual representation of real-world entities, their attributes, behaviors, and relationships.
- ▶ Plays a crucial role in designing structured, maintainable, and scalable software systems in OOP.

Key Benefits of Domain Models in OOP

- ▶ Clear Understanding of the Problem Domain
- ▶ Drives Object-Oriented Design (OOD)
- ▶ Improves Code Maintainability & Reusability
- ▶ Enhances Communication Between Stakeholders
- ▶ Reduces Complexity & Avoids Redundancy
- ▶ Aids in Database Design

Clear Understanding of the Problem Domain

- ▶ Helps visualize key objects and their relationships.
- ▶ Bridges the gap between business requirements and technical implementation.
- ▶ **Example:** In a Library Management System, objects like Book, Member, and Librarian are modeled clearly.

Drives Object-Oriented Design (OOD)

- ▶ The model naturally translates into OOP concepts like classes, objects, and relationships.
- ▶ **Example:**
 - ▶ Class → A real-world entity (Book, Student)
 - ▶ Attributes → Properties of the entity (title, author)
 - ▶ Methods → Behaviors of the entity (borrow(), returnBook())

Improves Code Maintainability & Reusability

- ▶ Leads to modular OOP code, making it easier to extend and refactor.
- ▶ **Example:** If a new type of book (eBook, AudioBook) needs to be added, the Book class can be extended using inheritance.

Enhances Communication Between Stakeholders

- ▶ Helps developers, designers, and business analysts speak a common language.
- ▶ **Example:** A UML class diagram derived from the domain model ensures all stakeholders understand the system.

Reduces Complexity & Avoids Redundancy

- ▶ Encourages encapsulation, cohesion, and low coupling in OOP.
- ▶ **Example:** Instead of repeating attributes in multiple classes, the domain model suggests composition (e.g., a LibraryCard object inside Member).

Aids in Database Design

- ▶ Maps naturally to a relational database schema or an object-relational mapping (ORM) system.
- ▶ **Example:**
 - ▶ The Student entity in the domain model translates to a Student table in an SQL database.
 - ▶ The borrowed_books relationship translates into a Many-to-Many table (student_id, book_id).

CRC Cards for Object-Oriented Software Design

Library Management and Attendance Management Systems

What are CRC Cards?

- ▶ CRC (Class-Responsibility-Collaboration) cards help in designing OOP systems.
- ▶ Each card represents a class and contains:
 - ▶ **Class Name** (Top)
 - ▶ **Responsibilities** (Left side) – What the class does
 - ▶ **Collaborators** (Right side) – Other classes it interacts with
- ▶ Useful for brainstorming and iterative design.

Why Use CRC Cards?

- ▶ Encourages collaborative and team-driven design.
- ▶ Simplifies early-stage object-oriented modeling.
- ▶ Helps identify potential issues before implementation.
- ▶ Supports Agile and UML-based workflows.

Library Management System Overview

- ▶ Manages books, members, and borrowing processes.
- ▶ Tracks availability, reservations, and due dates.
- ▶ Generates reports on book loans.

Identified Classes for Library System

- ▶ **Book** – Stores details, availability, reservations.
- ▶ **Member** – Registers, borrows books, pays fines.
- ▶ **Librarian** – Manages books and members.
- ▶ **Loan** – Handles borrowing, due dates, fines.

CRC Card for Book Class

Class: Book

► Responsibilities:

- Store book details.
- Check availability.
- Reserve a copy.

► Collaborators: Member, Librarian

CRC Card for Member Class

Class: Member

► Responsibilities:

- Register in system.
- Borrow books.
- Pay late fees.

► Collaborators: Book, Loan

Attendance Management System Overview

- ▶ Tracks student attendance for university courses.
- ▶ Allows faculty to mark attendance and generate reports.
- ▶ Admins can manage courses, students, and faculty.

Identified Classes for Attendance System

- ▶ **Student** – Enrolls in courses, views attendance.
- ▶ **Faculty** – Marks attendance, generates reports.
- ▶ **Admin** – Manages users and courses.
- ▶ **Course** – Stores student and faculty lists.
- ▶ **AttendanceRecord** – Stores attendance data.
- ▶ **Report** – Generates attendance reports.

CRC Card for Student Class

Class: Student

► Responsibilities:

- View attendance records.
- Enroll in courses.
- Check attendance percentage.

► Collaborators: Course, AttendanceRecord

CRC Card for Faculty Class

Class: Faculty

► Responsibilities:

- Mark student attendance.
- View and edit attendance.
- Generate reports.

► Collaborators: Course, AttendanceRecord, Report

CRC Cards Table

Class	Responsibilities	Collaborators
Student	- Enroll in courses - View attendance records - Check attendance percentage	Course, AttendanceRecord
Faculty	- Mark attendance - Generate reports - Modify records	Course, AttendanceRecord, Report
Admin	- Manage users - Assign faculty - Oversee policies	Faculty, Course
Course	- Manage enrollments - Link faculty - Store attendance data	Student, Faculty, AttendanceRecord
AttendanceRecord	- Store attendance status - Update records	Student, Faculty, Course
Report	- Generate attendance reports - Analyze trends	AttendanceRecord, Faculty

Key Benefits of Domain Models in OOP

- ▶ **Clear Understanding of the Problem Domain**

Domain models help visualize key objects and relationships.

- ▶ **Drives Object-Oriented Design (OOD)**

The model naturally translates into OOP concepts like classes and objects.

- ▶ **Improves Code Maintainability & Reusability**

Modular design makes it easier to extend and refactor.

- ▶ **Enhances Communication Between Stakeholders**

UML class diagrams ensure a shared understanding of the system.

- ▶ **Reduces Complexity & Avoids Redundancy**

Encourages encapsulation, cohesion, and low coupling.

- ▶ **Aids in Database Design**

Maps naturally to relational databases and ORM systems.

From CRC Cards to Code

- ▶ Convert responsibilities into class methods.
- ▶ Implement collaborations using method calls.
- ▶ Follow SOLID principles for maintainable code.

Conclusion

- ▶ Domain models play a critical role in OOP by improving structure, maintainability, and communication.
- ▶ Domain models help in designing robust and scalable software systems.
- ▶ CRC cards simplify object-oriented design.
- ▶ CRC Cards help in designing OOP systems effectively.
- ▶ Transitioning from CRC to code ensures modular, maintainable software.
- ▶ Clear responsibilities and collaborations improve maintainability.
- ▶ The Library and Attendance systems demonstrate structured modeling.